

Human-Aware Obstacle Avoidance Implementation Specification

Material Transfer Robot Project

Farmience

Version 1.0 | March 25, 2026

Table of Contents

Section	Page
1. Overview & Design Philosophy	3
2. Architecture	3
3. Implementation Phases	4
4. Phase 0: ZED Configuration	4
5. Phase 1: Human Costmap Layer	5
6. Phase 2: HumanBlockingPath BT Condition	6
7. Phase 3: RaiseHumanAlarm BT Action	7
8. Phase 4: WaitUntilHumanClears BT Action	8
9. Phase 5: Behavior Tree Integration	9
10. Phase 6: Tuning & Testing	10
11. Package Structure	11
12. Configuration Reference	11

1. Overview & Design Philosophy

This document specifies the obstacle avoidance strategy for the Material Transfer Robot, a floor-level delivery robot operating in indoor environments. The approach distinguishes between human obstacles (which trigger stop-and-alarm behavior) and static obstacles (which are navigated around using point cloud data).

1.1 Core Principles

- Human = Dynamic:** Any detected human is treated as a dynamic obstacle, regardless of whether they are moving or stationary.
- Stop & Alarm for Humans:** When a human blocks the robot's planned path and is within a threshold distance, the robot stops and raises an alarm rather than attempting to navigate around them.
- Navigate Around Static Obstacles:** All non-human obstacles detected via point cloud are added to the Nav2 costmap, and the robot navigates around them normally.
- Floor Cutoff:** Point cloud data below a threshold height (e.g., 5cm) is ignored to remove floor detections.

1.2 Decision Logic

The robot follows this decision flow when a human is detected:

- Is a human detected? If NO, continue normal navigation with point cloud obstacles
- If YES, add human to costmap as lethal obstacle
- Is human on the planned path? If NO, continue (human is off to the side)
- If YES, is robot within threshold distance (e.g., 1.5m)? If NO, keep moving (human may clear)
- If YES, STOP, RAISE ALARM, WAIT until human clears

2. Architecture

The system integrates with the existing Nav2 stack and ZED camera infrastructure.

2.1 Data Flow

Source	Topic	Consumer	Purpose
ZED Camera	/zed/point_cloud/cloud_registered	Nav2 ObstacleLayer	Static obstacle detection
ZED Camera	/zed/obj_det/objects	HumanCostmapLayer	Human position in costmap
ZED Camera	/zed/obj_det/objects	BT Plugins	Path blocking detection

2.2 Component Overview

Component	Type	Description
-----------	------	-------------

HumanCostmapLayer	Nav2 Costmap Plugin	Marks human positions as lethal obstacles with inflation
HumanBlockingPath	BT Condition Node	Returns SUCCESS if human is on path AND within threshold distance
RaiseHumanAlarm	BT Action Node	Publishes alarm status to topic/service
WaitUntilHumanClears	BT Action Node	Blocks until human no longer blocks path

Note: No separate human filter node is required. ZED SDK's `mc_*` parameters filter object detection classes at the source, publishing only person detections.

3. Implementation Phases

Phase	Description	Estimated Effort
0	ZED config update + verify people-only detection	30 min
1	Package scaffold + HumanCostmapLayer	3-4 hours
2	HumanBlockingPath BT condition	2-3 hours
3	RaiseHumanAlarm BT action	1 hour
4	WaitUntilHumanClears BT action	1-2 hours
5	BT integration + full navigation test	2-3 hours
6	Tuning + edge case testing	2-3 hours

Important: Each phase must be tested and verified before proceeding to the next.

4. Phase 0: ZED Configuration

4.1 Objective

Configure ZED object detection to publish only person detections, filtering out other classes (vehicles, bags, animals, electronics).

4.2 Configuration Changes

Add the following parameters to `robot_bringup/config/zedm.yaml` under the `object_detection` section:

```
object_detection:
  allow_reduced_precision_inference: true
  confidence_threshold: 50.0
  filtering_mode: 2
  max_range: 5.0
  model: MULTI_CLASS_BOX_FAST
  object_tracking_enabled: true
  od_enabled: true
  prediction_timeout: 3.0
  # ADD THESE - filter to people only
  mc_people: true
  mc_vehicle: false
  mc_bag: false
  mc_animal: false
  mc_electronics: false
```

4.3 Verification

1. Launch ZED node: `ros2 launch robot_bringup robot_bringup.launch.py`
2. Monitor detections: `ros2 topic echo /zed/obj_det/objects`
3. Walk in front of camera - Should see PERSON detections
4. Place bag/bottle in view - Should NOT appear in detections

Success Criteria: Only objects with label 'Person' are published. All other classes are filtered.

5. Phase 1: Human Costmap Layer

5.1 Objective

Create a Nav2 costmap layer plugin that subscribes to ZED object detection, extracts human positions, and marks them as lethal obstacles in the costmap with appropriate inflation.

5.2 Files to Create

File	Purpose
include/human_detection/human_costmap_layer.hpp	Header file with class declaration
src/human_costmap_layer.cpp	Implementation of costmap layer
human_costmap_layer_plugin.xml	Pluginlib export descriptor

5.3 Key Implementation Details

- Subscribe to /zed/obj_det/objects (type: zed_interfaces/msg/ObjectsStamped)
- For each detected human, project 3D bounding box to 2D ground plane
- Mark cells within human footprint as LETHAL_OBSTACLE (cost = 254)
- Apply inflation radius around human (configurable, default 0.8m)
- Clear human positions after persistence timeout (configurable, default 1.0s)
- Transform human positions from camera frame to costmap frame using TF2

5.4 Parameters

Parameter	Type	Default	Description
human_topic	string	/zed/obj_det/objects	Topic for human detections
inflation_radius	double	0.8	Personal space buffer (meters)
persistence_time	double	1.0	Time to keep human in costmap (s)
human_cost	int	254	Cost value for human cells

5.5 Verification

1. Launch full navigation stack with human layer enabled
2. Open RViz, display local_costmap
3. Walk in front of robot - Red (lethal) cells should appear
4. Walk away - Cells should clear after persistence timeout
5. Verify Nav2 controller naturally avoids human positions

6. Phase 2: HumanBlockingPath BT Condition

6.1 Objective

Create a Behavior Tree condition node that returns SUCCESS when a human is detected on the robot's planned path AND the robot is within a threshold distance of that human.

6.2 Logic Flow

```
tick():
1. Get current path from BT blackboard
2. Get robot's current pose (from TF: map -> base_link)
3. Get human detections from /zed/obj_det/objects
4. For each human:
  a. Check if human footprint intersects any path waypoint
    (within path_intersection_buffer, e.g., 0.3m)
  b. If intersects:
    - Calculate distance from robot to human
    - If distance <= blocking_distance_threshold:
      return SUCCESS (human IS blocking)
5. return FAILURE (no blocking human)
```

6.3 BT Ports

Port	Direction	Type	Description
path	Input	nav_msgs::msg::Path	Planned path from planner
distance_threshold	Input	double (default: 1.5)	Distance to trigger blocking
path_buffer	Input	double (default: 0.3)	How close to path = 'on path'
human_topic	Input	string	Topic for human detections

6.4 Path-Human Intersection Algorithm

For efficient intersection checking: (1) Project human 3D bounding box to 2D polygon on ground plane. (2) For each waypoint in path (can skip every N waypoints for efficiency), check if waypoint (x,y) falls within path_buffer distance of human polygon. (3) Use simple point-in-polygon or distance-to-polygon check. Optimization: Only check waypoints ahead of robot's current position, not the entire path.

6.5 Verification

- Create test BT that logs condition result
- Send navigation goal, stand in path at various distances
- At >1.5m on path - Should return FAILURE
- At <1.5m on path - Should return SUCCESS
- Standing to the side (not on path) - Should return FAILURE

7. Phase 3: RaiseHumanAlarm BT Action

7.1 Objective

Create a simple BT action node that triggers the alarm system when a human blocks the path.

7.2 Implementation

```
on_tick():
  1. Publish to /human_alarm/active (std_msgs/Bool = true)
  2. Optionally call alarm service (for buzzer/LED/UI notification)
  3. return SUCCESS
```

7.3 BT Ports

Port	Direction	Type	Description
alarm_topic	Input	string	Topic to publish alarm state (default: /human_alarm/active)

7.4 Service Definition (Optional)

```
# srv/TriggerAlarm.srv
bool activate
---
bool success
string message
```

8. Phase 4: WaitUntilHumanClears BT Action

8.1 Objective

Create a BT action node that blocks (returns RUNNING) until the human is no longer blocking the planned path. While waiting, it continuously publishes zero velocity for safety.

8.2 Implementation

```
tick():
  1. Check HumanBlockingPath condition (same logic as Phase 2)
  2. If human still blocking:
    - Publish zero velocity to /cmd_vel (safety override)
    - return RUNNING
  3. If human cleared:
    - Publish to /human_alarm/active (Bool = false)
    - return SUCCESS
```

8.3 BT Ports

Port	Direction	Type	Description
path	Input	nav_msgs::msg::Path	Planned path to check
distance_threshold	Input	double (default: 1.5)	Distance threshold

human_topic	Input	string	Topic for human detections
-------------	-------	--------	----------------------------

8.4 Verification

- Start navigation, stand in path within threshold
- Robot should stop, alarm should be active
- Action should return RUNNING continuously
- Move away from path
- Action should return SUCCESS, alarm should clear
- Robot should resume navigation automatically

9. Phase 5: Behavior Tree Integration

9.1 Objective

Integrate all BT plugins into the navigation behavior tree, working alongside existing tile-switching logic.

9.2 Behavior Tree Structure

```
<BehaviorTree ID="NavigateWithHumans">
  <PipelineSequence>

    <!-- Compute path -->
    <ComputePathToPose goal="{goal}" path="{path}"
                      planner_id="GridBased"/>

    <!-- Existing tile logic -->
    <TileCheck path="{path}" .../>

    <!-- Human-aware following -->
    <ReactiveFallback>
      <!-- Primary: Follow if no blocking human -->
      <ReactiveSequence>
        <Inverter>
          <HumanBlockingPath
            path="{path}"
            distance_threshold="1.5"
            human_topic="/zed/obj_det/objects"/>
        </Inverter>
        <FollowPath path="{path}" controller_id="FollowPath"/>
      </ReactiveSequence>

      <!-- Fallback: Stop and wait for human -->
      <Sequence>
        <RaiseHumanAlarm alarm_topic="/human_alarm/active"/>
        <WaitUntilHumanClears
          path="{path}"
          distance_threshold="1.5"
          human_topic="/zed/obj_det/objects"/>
        </Sequence>
      </ReactiveFallback>

    </PipelineSequence>
  </BehaviorTree>
```

9.3 Nav2 Params Update

Add the human layer to nav2_params.yaml:

```
local_costmap:
  local_costmap:
    ros__parameters:
      plugins: ["obstacle_layer", "human_layer", "inflation_layer"]

    obstacle_layer:
      plugin: "nav2_costmap_2d::ObstacleLayer"
      enabled: true
      observation_sources: zed_pointcloud
      zed_pointcloud:
        topic: /zed/point_cloud/cloud_registered
        data_type: "PointCloud2"
        min_obstacle_height: 0.05 # Floor cutoff
        max_obstacle_height: 1.0
        marking: true
        clearing: true

    human_layer:
```

```
plugin: "human_detection::HumanCostmapLayer"  
enabled: true  
human_topic: "/zed/obj_det/objects"  
inflation_radius: 0.8  
persistence_time: 1.0
```

10. Phase 6: Tuning & Testing

10.1 Parameters to Tune

Parameter	Range	Notes
distance_threshold	1.0m - 2.0m	How close before stopping
path_buffer	0.2m - 0.5m	Width of 'path corridor'
inflation_radius	0.5m - 1.0m	Personal space around humans
persistence_time	0.5s - 2.0s	How long human stays in costmap
min_obstacle_height	0.03m - 0.08m	Floor cutoff threshold

10.2 Test Scenarios

Scenario	Expected Behavior
No humans, navigate A to B	Normal navigation
Human standing to the side	Navigate past, no alarm
Human blocking path, far (>threshold)	Continue approaching
Human blocking path, close (<threshold)	Stop, raise alarm, wait
Human moves away while robot waiting	Clear alarm, resume navigation
Human walks across path ahead	Brief pause if close, then continue
Multiple humans, one blocking	Stop for blocking human only
Human detection flickers	No rapid stop/start (use persistence)

11. Package Structure

```
ros_ws/src/human_detection/
├── CMakeLists.txt
├── package.xml
├── config/
│   └── human_detection_params.yaml
├── include/human_detection/
│   ├── human_costmap_layer.hpp
│   ├── human_blocking_path.hpp
│   ├── raise_human_alarm.hpp
│   └── wait_human_clears.hpp
├── src/
│   ├── human_costmap_layer.cpp
│   ├── human_blocking_path.cpp
│   ├── raise_human_alarm.cpp
│   ├── wait_human_clears.cpp
│   └── bt_plugin_registration.cpp
├── srv/
│   └── TriggerAlarm.srv
├── human_costmap_layer_plugin.xml
├── launch/
│   └── human_detection.launch.py
```

12. Configuration Reference

12.1 human_detection_params.yaml

```
human_detection:
  ros__parameters:
    # Costmap layer parameters
    human_topic: "/zed/obj_det/objects"
    inflation_radius: 0.8
    human_cost: 254
    persistence_time: 1.0

    # BT parameters
    blocking_distance_threshold: 1.5
    path_intersection_buffer: 0.3

    # Alarm parameters
    alarm_topic: "/human_alarm/active"
    cmd_vel_topic: "/cmd_vel"
```

12.2 Dependencies

Package	Purpose
nav2_costmap_2d	Base class for costmap layer
pluginlib	Plugin registration
behaviortree_cpp_v3	BT node base classes
zed_interfaces	ObjectsStamped message type
tf2_ros	Coordinate frame transformations
geometry_msgs	Pose, Point, Polygon messages

--- End of Document ---